Министерство образования и науки Российской Федерации

Федеральное Государственное Автономное Образовательное

Учреждение Высшего Образования "Национальный

Исследовательский Университет ИТМО"

Факультет программной инженерии и компьютерной техники

Лабораторная работа №1

По дисциплине "Тестирование программного обеспечения"

Тема: "JUnit 5"

Вариант: 423925

Выполнил студент группы P3319

Волокитин Александр Сергеевич

Проверил преподаватель практики

Тюрин Иван Николаевич

г. Санкт-Петербург, 2026 г.

Условие

Лабораторная работа #1

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

Введите вариант: 423925

1. Функция arctg(x)
2. Программный модуль для работы с биномиальной кучей (Logical Representation, http://www.cs.usfca.edu/~galles/visualization/BinomialQueue.html)
3. Описание предметной области:

> Простетный Вогон Джельц перетаскивал свое неприятное зеленое тело по командному мостику. Он всегда бывал несколько раздражителен после уничтожения населенных планет. Ему хотелось, чтобы кто-нибудь пришел и сказал, что произошла ошибка, и тогда он наорал бы на этого кого-нибудь и ему стало бы лучше. Он со всей силой плюхнулся в кресло, в надежде, что оно сломается, и он сможет неподдельно разозлиться, но кресло лишь жалобно скрипнуло.

Задание 1

1. Область определения функции: $x \in R$
2. Область значений функции: $f(x) \in \left(\frac{-\Pi}{2}; \frac{\Pi}{2}\right)$
3. Горизонтальные асимптоты:
   - $$\lim_{x \to \infty} f(x) = \frac{\Pi}{2}$$
   - $$\lim_{x \to -\infty} f(x) = \frac{-\Pi}{2}$$
4. Особые точки
   - $f(x) = 0$ (точка перегиба)

Тестирование проверяет корректность реализации функции арктангенса через ряд Тейлора и обработку аргументов командной строки.

Тест 1 (shouldCalculateArctanWithDefaultIterations):

Проверка вычисления арктангенса со стандартным количеством итераций (100). Результат сравнивается со встроенной функцией atan().

Тест 2 (shouldCalculateArctanWithCustomIterations):

Проверка вычисления арктангенса с заданным количеством итераций (200) через аргумент командной строки.

Тест 3 (shouldThrowExceptionWhenNoArguments):

Проверка обработки ошибки при запуске программы без аргументов — должно выбрасываться исключение с сообщением “Provide an argument to a program”.

Тест 4 (shouldConvergeWithMoreIterations):

Проверка сходимости ряда — увеличение количества итераций должно давать более точный результат (ближе к эталонному atan()).

Тест 5 (shouldCalculateRandomValue):

Стресс-тест на 1 000 000 случайных значений для проверки корректности работы функции на всём диапазоне входных данных.

Задание 2

Тестирование проверяет корректность реализации биномиальной кучи — структуры данных, реализованной на основе множества биномиальных деревьев и поддерживающей эффективные операции вставки и извлечения экстремума (минимума в конкретно этой реализации).

Тест 1 (shouldReturnEmptyHeap):

Проверка поведения пустой кучи. Метод top() должен возвращать null, метод extractTop() — null.

Тест 2 (shouldHandleSingleNode):

Проверка базовой функциональности на одном элементе. После вставки значения top() и extractTop() возвращают это значение, после извлечения куча становится пустой.

Тест 3 (shouldSortElements):

Проверка корректности извлечения элементов в порядке возрастания. После вставки 100 000 случайных значений результаты последовательного извлечения сравниваются с отсортированным исходным списком. Конечное состояние кучи — пустое.

Задание 3

Тестирование проверяет логику перехода объектов из одного состояния в другое согласно текстовому описанию предметной области.

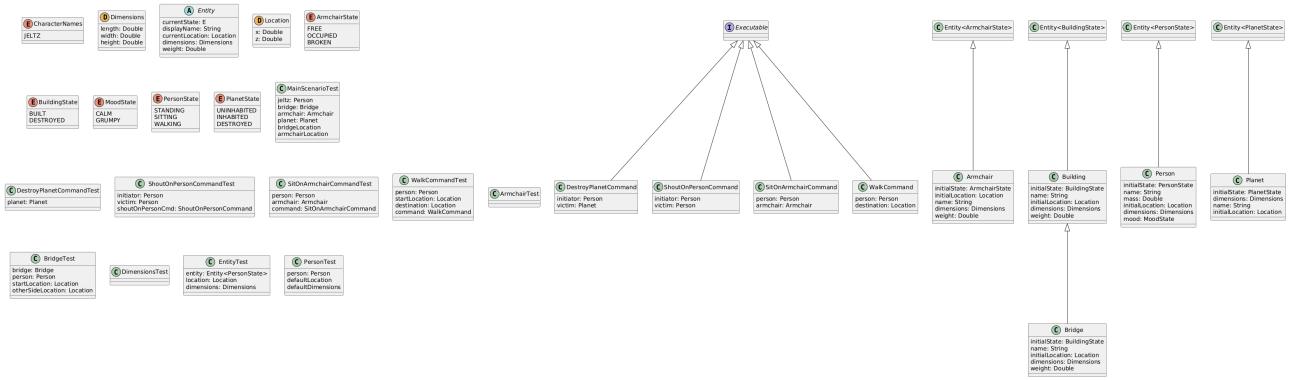

Figure 1: UML-диаграмма классов для задания 3

Тестирование предметной области

Набор тестов проверяет корректность выполнения команд и состояний объектов в модели взаимодействия персонажей.

Команда SitOnArmchairCommand (shouldTakeFreeArmchair):

Проверка успешного занятия свободного кресла — персонаж переходит в состояние SITTING, кресло отмечается как OCCUPIED, владелец кресла фиксируется.

Команда ShoutOnPersonCommand (shouldChangeMoodAfterShouting):

Проверка изменения настроения при команде “крикнуть” — инициатор из GRUMPY переходит в CALM, жертва становится GRUMPY.

Команда WalkCommand (shouldWalkToDestination):

Проверка перемещения персонажа в указанную точку — после выполнения команды персонаж оказывается в целевой локации, состояние остается STANDING.

Модель Person (shouldCreatePersonWithNameConstructor):

Проверка создания персонажа через конструктор с именем — устанавливаются значения по умолчанию: состояние STANDING, вес 80.0, стандартные размеры, локация (0,0), настроение CALM.

Модель Bridge (shouldCrossBridgeFromStartToOtherSide):

Проверка перехода персонажа по мосту — после пересечения моста персонаж оказывается на противоположной стороне, его состояние остается STANDING.

Вывод

В ходе выполнения лабораторной работы я изучил базовые возможности JUnit 5 на примере разработки модульного тестового покрытия для программ трёх типов: математическая функция, алгоритм и сценарий на базе описанной предметной области.